

Anotaciones 29-08-25

Sistemas Distribuidos

Single Point of Failure y Alta Disponibilidad

Para que un sistema tenga alta disponibilidad, es importante evitar el Single Point of Failure a nivel de hardware (nivel físico):

Los dispositivos blade server son de uso común. Se miden por medio de "US", una unidad que indica la altura o el ancho del dispositivo, es decir, la cantidad de ranuras donde se pueden meter discos. El RAID Controller toma todos los discos insertados y permite que funcionen en conjunto, y se comportan como un solo disco.

Para asegurar la alta disponibilidad a nivel de disco, los discos se pueden comportar como "espejo", para agregar redundancia en caso de que los discos principales tengan algún tipo de fallo. Si esto llegara a suceder, se hace un fail over automático, y el control pasa a los discos que estaban reflejando a los discos principales, para que el sistema pueda seguir funcionando mientras se recupera el fallo. En este caso la capacidad del Sistema Operativo será la mitad que el total de discos, ya que la otra mitad está destinada a ser un reflejo de la principal.

A nivel de procesador, normalmente estos dispositivos tienen más de un procesador. Así, si se tiene un fallo en un procesador, el control lo puede tomar el otro procesador y el servicio va a seguir funcionando.

A nivel de memoria ocurre lo mismo. Se tiene redundancia a nivel de memoria para recuperación en caso de fallos. Si por ejemplo hay 64GB de RAM instalados, el Sistema Operativo asume que el máximo que puede usar son 32GB, ya que los otros 32GB están en espera para tomar el control en caso de fallos.

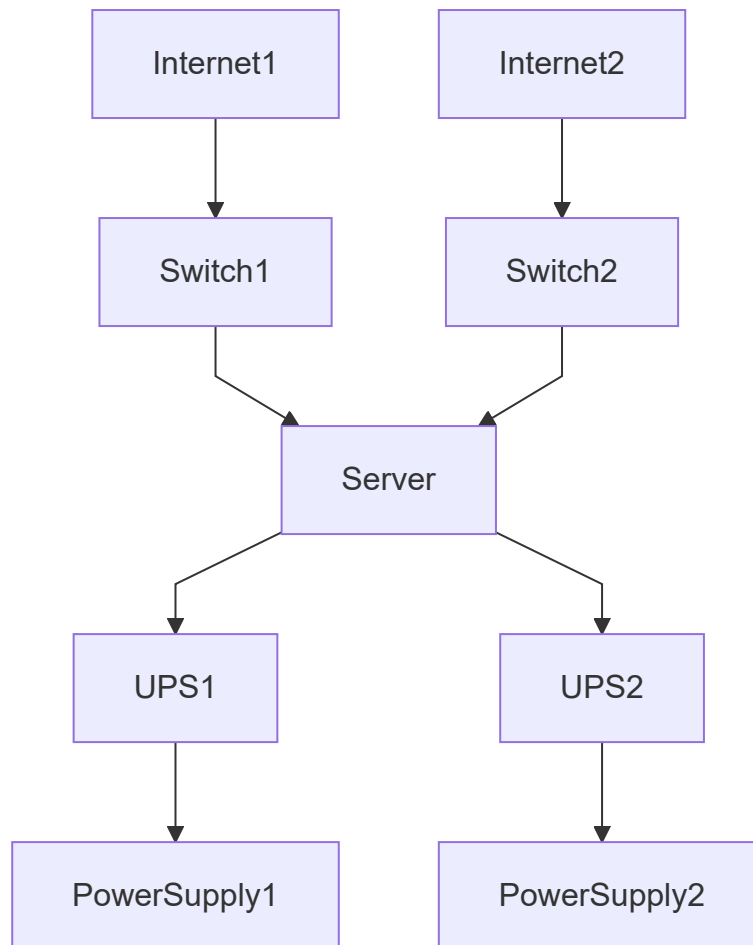
A nivel de fuentes de poder, (server power supply), normalmente se tienen dos, se meten en el servidor y están conectadas a la alimentación. Si una se daña, se cambia por una nueva, y el sistema automáticamente usa la fuente de poder que está funcionando. Cuando se dañan los discos sucede lo mismo. Se cambian los discos dañados y la información se reconstruye automáticamente con los discos que estaban reflejando la información.

El hardware siempre tendrá una duplicidad o redundancia de componentes, para asegurar que siempre haya alta disponibilidad. Además, el hardware tiene un sistema de observabilidad, donde puede detectar la deterioración incluso años antes de que deje de funcionar.

El servidor también debe tener redundancia a nivel de red

Se deben tener tarjetas de red independientes, que a su vez deben estar conectadas con switches independientes. También deben estar conectados a power supply independientes, que deben tener proveedores redundantes e independientes de electricidad (por ejemplo, el ICE y la JASEC, que son proveedores separados). También es común que se tengan UPS, las cuales son baterías que permiten que la computadora trabaje por un tiempo sin conexión a internet y evita picos de corriente. Usualmente el UPS se encuentra en medio de la conexión del servidor y la corriente para absorber los picos de energía y reducir la posibilidad de que se queme la tarjeta de red. También se hace failover automático.

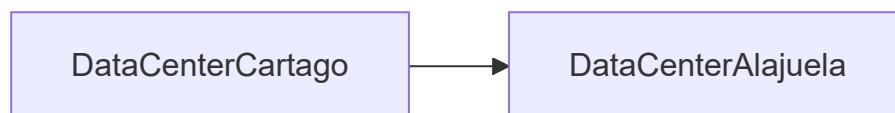
También se debe tener redundancia a nivel de proveedor de internet



Un caso extremo de redundancia sería tener otro sistema igual que funcione como espejo. Esto nos daría un sistema altamente disponible y distribuido. Sin embargo, esto no siempre se hace, ya que es sumamente costoso. Es importante asegurarse que los sistemas no estén en un solo rack, ya que si el rack es el mismo se convierte en un single point of failure.

Se configura algo llamado Zone Awareness o Allocation Awareness para colocar las bases de datos en servidores distintos o servidores en lugares específicos, lo cual nos puede ayudar a evitar un Single Point of Failure si las bases de datos principal y secundaria están en el mismo servidor.

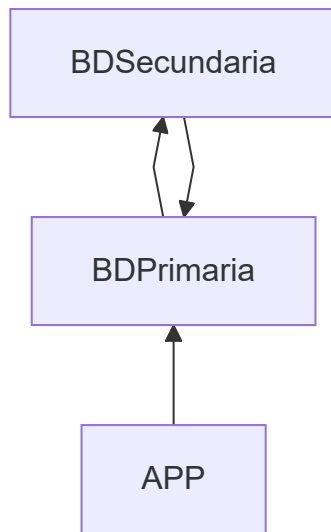
Una vez que tenemos redundancia a nivel de hardware y a nivel geográfico, también se necesita redundancia a nivel de Data Center. No sirve de nada tener toda la redundancia anterior si todo está en un mismo Data Center, y el Data Center se cae, entonces todo el sistema se caería.



Hay que asegurarse que todo tenga redundancia desde Data Center hasta la base de datos. Después de esto, hay que asegurarse que el cliente pueda utilizarla.

Data Tiers

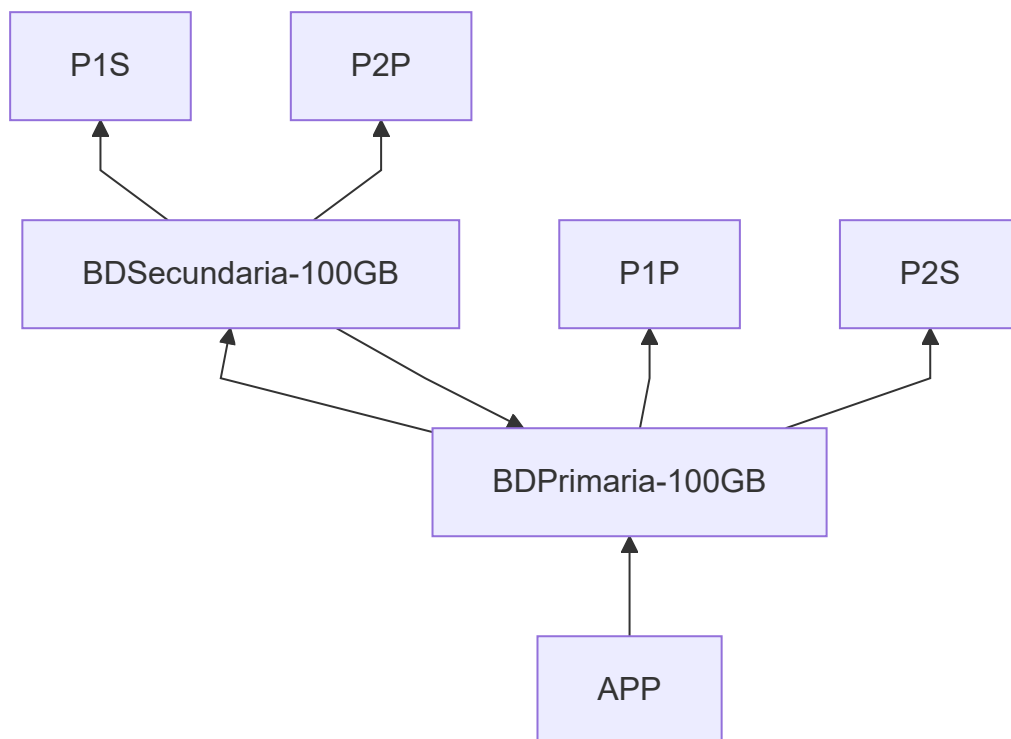
Se ve el concepto de **Partitions** y **Shards** (chunks, slices).



Normalmente, la aplicación está conectada a la Base de Datos Primaria. Si esta se cae, la secundaria toma el rol de primaria y la app se conecta a esta.

El problema que tenemos es que **al duplicar los recursos disponibles, la mitad de esos recursos se están desperdiciando**. Para resolver este problema, podemos hacer particiones al DataSet.

Partition 1 (P1)	Partition 2 (P2)
50GB	50GB



En vez de desperdiciar los recursos, puedo hacer particiones tanto de la primaria como la secundaria e instruimos al motor de base de datos a hacer una distribución. No es necesario que la partición primaria y secundaria queden en el mismo servidor. Las queries se pueden enviar a particiones específicas para ser ejecutadas. Sin embargo, como ambas BD dan un Result Set, hay que hacer una unión de ambos al enviar la

respuesta.

A partir de esto surgen Data Tiers

Existe un concepto llamado **ratio**. El ratio indica la relación entre GB de memoria por GB de disco. Por ejemplo, HOT tiene 1 de RAM por 30 de disco. WARM tiene 1 de RAM por 60 de disco. COLD tiene 1 de RAM por 120 de disco. FROZEN tiene 1 de RAM por + 120 de disco.

Entre más cercano este los GB de RAM y del disco, aumenta la cantidad de Caché Hits, y gracias a esto se dan menos Context Switches, lo cual mejora el rendimiento. Lo ideal sería tener 1:1, pero esto puede llegar a ser muy costoso. Empresas como Amazon tienen relaciones de 1:1, o negocios pequeños que tienen catálogos pequeños pero necesitan realizar consultas muy rápidamente.



Para aprovechar mejor los recursos, podemos hacer algo llamado Data Tiers. Esto consiste en asignar ciertas consultas a diferentes servidores. Si identificamos que ciertas consultas son menos comunes, podemos asignarlas a un servidor más lento (FROZEN, COLD), lo cual supone una menor inversión. Si las consultas son muy comunes se las asigna a un servidor más rápido (WARM, HOT) los cuales requieren una mayor inversión.

Por ejemplo, en el caso de el portal donde los estudiantes ingresan a consultar sus notas, podemos tener la siguiente configuración:

- $id < 2000 = \text{FROZEN}$
- $id < 2008 = \text{COLD}$
- $id < 2015 = \text{WARM}$
- $id > 2015 = \text{HOT}$

Compliance

A nivel de Bases de Datos, hay ciertas legislaciones internacionales que hay que cumplir.

Por ejemplo, para asegurar la protección de datos sensibles de sus habitantes, gobiernos de Europa tienen algo llamado General Data Protection Regulation (GDPR), donde prohíben que datos de europeos sean guardados en bases de datos ubicadas geográficamente fuera de Europa. Así que si creo un sistema de base de datos ubicado en Estados Unidos Virginia y quiero almacenar datos de europeos allí, estaría incumpliendo el GDPR.

Para mantener la compliance, se pueden hacer particiones basadas en la ubicación de la persona. Con esto, se puede usar Zone Awareness con una distribución de servidores en ubicaciones geográficas. Así si un usuario es de la Unión Europea, su información no quedará guardada fuera de la Unión Europea.

Data Networks

Hoy en día este concepto es relacionado con cloud providers. En los sistemas distribuidos se suele tener una red. La red utiliza un medio (parte física) para conectarse.

Si la red está saturada o caída, hay un retraso en el envío de información. Si está muy saturada se empieza a notar un delay en las respuestas a los usuarios. Para asegurarse de que no está saturada, se tiene algo llamado el bandwidth: cantidad de datos que pueden pasar por unidad de tiempo.

Se tiene un Storage Area Network (SAN). Es muy potente, con una capacidad y tolerancia a fallos muy alta. Los SAN (también llamado NAS) se conectan en la red. De esta manera, puede almacenar los datos de las bases de datos en el storage.

Si las bases de datos están muy cargadas, se requiere más bandwidth, y se satura cada vez más. Aunque el storage y los servidores funcionan correctamente, si la red está saturada no habrá un buen rendimiento.

Man in the Middle: Si llega a hackear la red, ve la red y lo que pasa en la red. Es peligroso porque se puede robar información de esta manera. Las redes más sofisticadas separan la red de datos para protegerla de hackers, y la otra red atiende clientes. Así si entra un Man in the Middle, los datos estarán protegidos.

Load Testing

Se simula el workload (imitando el comportamiento que tendrían los usuarios). Si probamos diferentes casos límite del workload, podemos saber cuales son los límites del sistema y cuando puede fallar.

De esta manera, podemos programar alertas, que nos avisen si el workload del sistema sobrepasa los límites. De esta manera sabemos que se puede caer el sistema, y podemos hacer autoscaling para poder manejar el alto workload.

Object Storage

Conseguir memoria puede ser muy caro. No se puede tener toda la información en el disco duro. El object storage se asocia con los data tiers, ya que el object storage es un servicio de intercambio de archivos utilizando http, y los data tiers lo implementan.

Cuando un disco duro está conectado a la computadora, el disco duro se conecta directamente. Hay un formato de datos que facilita la traducción de datos al recuperarlos. Es costoso.

Con el servicio de intercambio de archivos, en cambio, se hace por medio de la red usando internet. Se compete por los recursos para traer los datos desde otra ubicación. Hay que traer el archivo completo en vez de solo una página. Esto lo hace lento comparado a un disco, pero es mejor para almacenamiento masivo con alta disponibilidad.

Read Replica

Es común que ahora la base de datos secundaria se use para queries de lectura. Esto quiere decir que está online al igual que la principal. La principal se encarga de todas las updates, y actualiza la replica. El load balancer recibe las queries y decide a qué base de datos debe enviarlas.

Esto es muy útil debido a que en la mayoría de sistemas, se hacen muchas más lecturas que updates (por ejemplo, en TikTok un solo video puede tener millones de visualizaciones), así que con este esquema se puede manejar una mayor cantidad de usuarios.

Multimaster

Puede ocurrir que en algún momento se tenga más de una master. Esto es un problema, debido a que el load balancer intentará enviar las queries a ambos servidores. Al hacer los updates, pueden intentar modificar simultáneamente ambos servidores, lo cuál posiblemente aumentará el riesgo de tener inconsistencias en los datos.

Cuando se da esto, es necesario que haya una elección de leader y consenso. Sin embargo, como tomará muchos mensajes ponerse de acuerdo, el cliente podrá notar un delay en el servicio.

Bare Metal

Se instala la base de datos directamente en el hardware. Es necesario encargarse del mantenimiento del hardware. Se da el concepto de On-Premise.

El uso de bare metal se redujo significativamente cuando aparecieron los cloud providers.

Colocation: Se contrata un data center. Esta ubicación tiene cubierta la seguridad y configuración de red, y el cliente instala sus cosas en el rack.

IaaS (Infrastructure as a Service)

El cliente compra la red, pero se encarga de su mantenimiento y el de la máquina virtual. (Ej: AWS, GCP, Azure).

Managed Services

El cliente le paga a un cloud provider, pero él mismo se encarga de instalar, configurar, dar mantenimiento y monitorear su servidor.

PaaS (Platform as a Service)

Se paga a una plataforma (como AWS), un servicio de Base de Datos. El cliente especifica la memoria, el disco y CPU que requiere, y la compañía solo le entrega un connection string. El cliente no debe preocuparse por nada de la configuración, solo utiliza la plataforma.

SaaS (Software as a Service)

Por ejemplo, una aplicación como Perusall: Por debajo tiene programación, Base de Datos, etc. Pero el cliente solo ve el resultado final, por ejemplo, se le permite asignar lecturas a los estudiantes, sin preocuparse por configurar nada.